

# GPUMODE Lecture Study Notes: Lecture 106: Hugging Face Kernels

## Core Problem the Lecture Addresses

The lecture explains the design of the **Hugging Face Kernels** project: a framework for building, distributing, and consuming machine learning kernels through a standardized hub-based workflow.

The central problem is that modern machine learning inference increasingly depends on **specialized hardware-specific kernels**. These kernels may be written in CUDA, Metal, ROCm, XPU, Triton, or other backend-specific systems. They are often required for fast inference in systems such as text generation servers, diffusion pipelines, and transformer libraries.

However, shipping these kernels is painful because kernel distribution combines several hard problems:

- **Native code distribution:** binaries must match the user's operating system, CPU architecture, Python ABI, PyTorch version, backend, CUDA version, GPU capability, and C++ ABI.
- **Large build matrices:** a single kernel may need variants across many combinations of PyTorch, CUDA, ROCm, XPU, Linux, macOS, Windows, x86, ARM, and GPU compute capability.
- **Long build times:** kernels such as FlashAttention can take significant time and memory to compile, especially because of heavy template instantiation.
- **Dependency conflicts:** different kernels may require different versions of CUDA, PyTorch, compiler flags, CUTLASS, or other build-time dependencies.
- **Non-standard source layouts:** every upstream kernel project tends to have its own directory structure, build scripts, wrapper code, and output naming scheme.
- **Non-standard binary outputs:** compiled shared libraries are often named in ways that do not clearly encode what platform or hardware they target.

- **Consumer friction:** downstream users should not need to understand build matrices, CUDA compatibility, glibc symbols, or PyTorch extension internals merely to call a faster operator.
- **Security risk:** distributing arbitrary compiled C++ or CUDA code creates supply-chain risks similar to, and in some ways stronger than, Python package distribution.

The high-level solution is to standardize the whole lifecycle:

kernel source  $\longrightarrow$  reproducible builder  $\longrightarrow$  versioned kernel variants on the hub  $\longrightarrow$  Python client resolution and download

The speaker compares this to shipping containers. A container standardizes how goods are packed, transported, and unloaded. Similarly, Hugging Face Kernels standardizes how kernels are built, uploaded, resolved, downloaded, and called.

The mental model is: build once correctly, publish variants in a predictable structure, and let many downstream applications consume the correct binary automatically.

## Required Background

### Machine Learning Kernels

A **kernel** is a low-level implementation of an operation such as attention, activation, normalization, quantization, matrix multiplication, sampling, or image processing.

In the machine learning systems context, a kernel is often designed to improve one or more of the following:

- reduce global memory traffic;
- increase arithmetic intensity;
- use tensor cores;
- fuse multiple operations into one launch;
- avoid intermediate tensor materialization;
- exploit a specific GPU architecture;
- specialize to model-specific shapes or semantics.

A typical PyTorch model may be written in high-level tensor operations, but fast production inference often depends on custom kernels for bottleneck operations.

## PyTorch Extensions

PyTorch supports native extensions through C++ and CUDA. A custom extension usually provides:

- C++ binding code that exposes a function to Python;
- CUDA or backend-specific implementation files;
- Python wrapper code;
- build scripts, often through `setuptools`, `torch.utils.cpp_extension`, CMake, Ninja, or custom scripts.

The simplified flow is:

Python call  $\rightarrow$  PyTorch binding  $\rightarrow$  C++ extension  $\rightarrow$  CUDA or backend kernel  $\rightarrow$  GPU execution.

The hard part is not only writing the kernel. It is building and distributing the extension in a way that works for many users.

## Ahead-of-Time Compilation Versus JIT Compilation

The lecture contrasts **ahead-of-time** compilation and **just-in-time** compilation.

**Ahead-of-time compilation** produces binaries before the user runs the program. This has several advantages:

- startup and warm-up are faster;
- users avoid local compilation;
- deployment containers can include ready-to-load kernels;
- production systems can validate binaries in CI.

However, AOT compilation creates a large build matrix:

$$N_{\text{variants}} = N_{\text{torch}} \cdot N_{\text{backend}} \cdot N_{\text{backend version}} \cdot N_{\text{OS}} \cdot N_{\text{arch}} \cdot N_{\text{ABI}} \cdot N_{\text{GPU capability}}.$$

**JIT compilation**, as used by Triton or other dynamic code-generation systems, compiles closer to runtime. This can adapt to newer hardware or specific shapes, but the trade-off is warm-up latency.

For a long-running training job, JIT warm-up may be acceptable. For a serverless or quickly scaled inference deployment, long JIT compilation can be unacceptable because the container must start serving quickly.

## GPU Architecture Concepts

Understanding the lecture requires the following GPU concepts.

### Streaming Multiprocessors

An NVIDIA GPU is organized into **streaming multiprocessors**, usually abbreviated as SMs. Each SM contains warp schedulers, registers, shared memory, load/store units, CUDA cores, and often tensor cores.

A kernel launch creates a grid of thread blocks:

$$\text{grid} = \{\text{block}_0, \text{block}_1, \dots, \text{block}_{B-1}\}.$$

Each block is scheduled onto an SM. Within each block, threads execute in warps of usually 32 threads.

### Memory Hierarchy

The relevant memory hierarchy is:

registers  $\rightarrow$  shared memory  $\rightarrow$  L1 cache  $\rightarrow$  L2 cache  $\rightarrow$  HBM/global memory.

A kernel is often fast when it minimizes traffic to global memory and maximizes reuse in registers, shared memory, and caches.

### Arithmetic Intensity

Arithmetic intensity is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

A kernel with low arithmetic intensity is usually memory-bound. A kernel with high arithmetic intensity may become compute-bound if it can use GPU compute units efficiently.

### Binary Compatibility

Native extensions must match or tolerate the user's system. Several compatibility dimensions matter.

## Python ABI

If an extension is tied to a specific Python version, then a separate build may be needed for Python 3.9, 3.10, 3.11, and so on. Python ABI3 can reduce this matrix by allowing one binary to work across multiple compatible Python versions.

## C++ ABI

Older PyTorch versions were compiled with different C++ ABIs, including the older C++98 ABI and the newer C++11 ABI. A binary compiled against one ABI may not load correctly against the other. The lecture notes that PyTorch later standardized on the C++11 ABI, reducing the variant count.

## glibc Symbol Versions

glibc uses versioned symbols. If a binary is compiled against a newer glibc, it may reference symbols unavailable on older systems. Therefore, binaries built on a new Linux distribution may fail on old production machines.

If a binary is compiled against an older glibc, it is more likely to run on newer systems:

old glibc target  $\longrightarrow$  compatible with many newer glibc versions.

This is why many projects build Linux wheels in old container images. The Hugging Face Kernels builder instead aims to use a modern toolchain while linking against an old compatibility target.

# Main Concepts

## The Three Components

The project has three main pieces:

1. **Kernel builder:** takes standardized source plus configuration and produces build variants.
2. **Hub:** stores kernel repositories and their variants.
3. **Client library:** resolves, downloads, validates, imports, and exposes kernels to user code.

The lifecycle is:

source tree  $\rightarrow$  builder  $\rightarrow$  variant directories  $\rightarrow$  hub repository  $\rightarrow$  `get_kernel`  $\rightarrow$  loaded Python module.

## Motivations From TGI, Transformers, and Diffusers

The project emerged from pain in systems such as **Text Generation Inference**, **Transformers**, and **Diffusers**. These projects need to ship fast and correct models across a broad user base.

For inference systems, specialized kernels are often not optional. They determine whether a model can meet latency, throughput, and memory requirements.

The problem is that each kernel historically required its own custom build process. One engineer might become the only person who knows how to build a particular kernel. This creates fragile tribal knowledge:

kernel expertise localized to one person → maintenance bottleneck → slower shipping.

The Kernels project attempts to replace bespoke workflows with a common format.

## Standardization of Build Input

A kernel repository has a standardized source structure. Important files and directories include:

- `build.toml`: declarative top-level configuration;
- `flake.nix`: connects the directory to the Nix-based builder;
- `torch-ext`: PyTorch binding and wrapper code;
- backend-specific source directories such as CUDA, Metal, ROCm, XPU, or CPU code.

The `build.toml` does not describe imperative steps. Instead, it declares meta-data, backends, source files, dependencies, and output expectations.

This distinction is important:

imperative build = “run these arbitrary steps”,

whereas

declarative build = “here are the inputs, outputs, and constraints”.

A declarative build format makes it easier for the builder to preserve reproducibility and standard output structure.

## Standardization of Build Output

The builder emits a `build` directory containing one subdirectory per variant.

A **variant** is a compiled artifact for a specific compatibility target. Conceptually:

variant = (PyTorch version, backend, backend version, OS, CPU architecture, ABI).

For CUDA, GPU compute capabilities are also relevant:

CUDA variant = (torch, CUDA, Linux, x86\_64, ABI, SM capability set).

A variant directory contains files such as:

- `ops.py`: generated Python glue that maps the binary to Python;
- `metadata.json`: metadata used for hub display and filtering;
- compiled shared object files such as `.so`, `.dylib`, or equivalent backend artifacts.

This makes the binary self-describing enough for automated resolution.

## Standardization of Consumption

The consumption side is designed to be simple. A user should be able to fetch a kernel from the hub and call it as a Python object.

Representative usage:

```
from kernels import get_kernel

activation = get_kernel(
    "kernels-community/activation",
    version=1
)

out = activation.silu_and_mul(x)
```

The user does not manually select CUDA version, glibc compatibility, operating system variant, or PyTorch extension file. The client library detects the environment and selects the best matching variant.

## Drop-In Layer Replacement

The project supports replacing full model layers, not only calling individual kernel functions. This matters because large libraries such as Transformers do not want model definitions filled with explicit `get_kernel` calls.

The goal is separation of concerns:

model architecture  $\neq$  kernel distribution logic.

A model can expose a layer as kernel-replaceable, and a mapping can later decide which hub kernel should implement its forward pass.

## Hardware-Level Explanation

### Why Kernels Matter for Inference

Inference systems often spend most of their time in a small number of kernels:

- attention;
- matrix multiplication;
- normalization;
- activation functions;
- sampling and logits processing;
- quantized dequantization plus GEMM;
- mixture-of-experts routing and dispatch;
- image or diffusion-specific operators.

The total inference latency can be approximated as:

$$T_{\text{step}} = \sum_{i=1}^K T_{\text{kernel},i} + T_{\text{launch}} + T_{\text{framework overhead}} + T_{\text{communication}}.$$

A custom fused kernel can reduce both the sum of kernel runtimes and the number of launches.

For example, a SiLU-and-multiply activation may be expressed as:

$$y = \text{SiLU}(a) \odot b,$$

where

$$\text{SiLU}(a) = a \cdot \sigma(a), \quad \sigma(a) = \frac{1}{1 + e^{-a}}.$$

Naively, this may create intermediate tensors:



$$a \rightarrow \sigma(a) \rightarrow a \cdot \sigma(a) \rightarrow (a \cdot \sigma(a)) \odot b.$$

A fused kernel computes the result in one pass:

$$y_i = \left( a_i \cdot \frac{1}{1 + e^{-a_i}} \right) b_i.$$

This reduces memory traffic from multiple reads and writes to approximately one read of  $a_i$ , one read of  $b_i$ , and one write of  $y_i$ .

### Memory Traffic for a Fused Activation

Assume FP16 inputs and output. Each element is 2 bytes.

A fused SiLU-and-multiply kernel has approximate global memory traffic:

$$B_{\text{fused}} = 2N + 2N + 2N = 6N \text{ bytes.}$$

A multi-stage unfused implementation may materialize intermediate tensors. If it writes and reads two intermediates, traffic may look like:

$$B_{\text{unfused}} \approx \underbrace{2N}_{\text{read } a} + \underbrace{2N}_{\text{write sigmoid}} + \underbrace{2N}_{\text{read sigmoid}} + \underbrace{2N}_{\text{read } a} + \underbrace{2N}_{\text{write silu}} + \underbrace{2N}_{\text{read silu}} + \underbrace{2N}_{\text{read } b} + \underbrace{2N}_{\text{write } y} = 16N \text{ bytes.}$$

The traffic reduction factor is approximately:

$$\frac{B_{\text{unfused}}}{B_{\text{fused}}} = \frac{16N}{6N} \approx 2.67.$$

For bandwidth-bound elementwise operations, this can translate directly into speedup.

### Kernel Distribution and GPU Architecture

A compiled CUDA extension may contain device code for specific SM capabilities. For example, a kernel may support:

$$\text{SM set} = \{75, 80, 86, 89, 90, 100, 120\}.$$

A kernel built only for Hopper-specific instructions may require:

$$\text{SM} \geq 90.$$

A FlashAttention-3 style kernel may target Hopper because it depends on architecture-specific instructions, memory pipelines, or tensor-core behavior. This is why the builder needs to encode hardware capability.

The client must answer:

Can this binary run on the current GPU?

That requires checking at least:

backend, CUDA compatibility, compute capability, OS, architecture, PyTorch ABI.

## CUDA Version Compatibility

The lecture emphasizes that CUDA compatibility has multiple layers.

There is a difference between:

- the **CUDA driver** and driver library;
- the **CUDA runtime and libraries** such as cuBLAS, cuDNN, or other CUDA toolkit components;
- the compiled device code and PTX/SASS generated by `nvcc` and `ptxas`.

It may be possible to run a CUDA 12.6-compiled kernel on a system with a CUDA 12.9-capable driver. But using a binary compiled against one toolkit with incompatible newer libraries can fail.

A useful simplified compatibility model is:

new driver  $\supseteq$  support for older CUDA runtime versions,

but

old compiled extension  $\nRightarrow$  universal compatibility with all future CUDA libraries.

The Hugging Face Kernels client can backtrack to an older CUDA variant if the exact version is unavailable. For example, if the user has CUDA 12.8 and the hub has a CUDA 12.6 compatible variant, it may choose the older variant if compatibility rules allow it.

# Mathematical Reasoning

## Build Matrix Growth

Let:

$N_T$  = number of PyTorch versions,

$N_B$  = number of backends,

$N_V$  = number of backend toolkit versions,

$N_O$  = number of operating systems,

$N_A$  = number of CPU architectures,

$N_C$  = number of GPU capability groups,

$N_{ABI}$  = number of ABI choices.

Then the naive number of build variants is:

$$N_{\text{variants}} = N_T N_B N_V N_O N_A N_C N_{ABI}.$$

If older PyTorch versions require two C++ ABIs, then:

$$N_{ABI} = 2.$$

If the ecosystem standardizes on one ABI, then:

$$N_{ABI} = 1,$$

and the matrix is cut in half:

$$\frac{N_T N_B N_V N_O N_A N_C \cdot 1}{N_T N_B N_V N_O N_A N_C \cdot 2} = \frac{1}{2}.$$

Similarly, Python ABI3 can reduce Python-version multiplication. If a project otherwise builds for  $P$  Python versions, ABI3 can ideally reduce:

$$N_{\text{python variants}} = P \rightarrow 1.$$

## Amortization Argument

The lecture's core economic argument is asymmetric cost.

Let:

$C_{\text{build}}$  = cost to build and validate a kernel variant,

$C_{\text{use}}$  = cost for a user to download and load a kernel,

$M$  = number of downstream uses.

Without standardized binary distribution, users may repeatedly build locally:

$$C_{\text{total, local}} = MC_{\text{build}}.$$

With centralized AOT builds:

$$C_{\text{total, hub}} = C_{\text{build}} + MC_{\text{use}}.$$

The standardization pays off when:

$$C_{\text{build}} + MC_{\text{use}} < MC_{\text{build}}.$$

Rearranging:

$$M > \frac{C_{\text{build}}}{C_{\text{build}} - C_{\text{use}}}.$$

Because  $C_{\text{build}} \gg C_{\text{use}}$ , this threshold can be small. For popular kernels such as FlashAttention, the amortization is extremely favorable.

## Variant Resolution as a Ranking Problem

The client performs variant resolution. Let the set of all available variants be:

$$\mathcal{V} = \{v_1, v_2, \dots, v_n\}.$$

Let the current environment be:

$$e = (\text{torch}, \text{backend}, \text{backend version}, \text{OS}, \text{arch}, \text{ABI}, \text{device capability}).$$

The client first filters:

$$\mathcal{V}_{\text{compatible}} = \{v \in \mathcal{V} \mid \text{compatible}(v, e)\}.$$

Then it sorts by priority:

$$v^* = \arg \max_{v \in \mathcal{V}_{\text{compatible}}} \text{priority}(v, e).$$

The lecture describes two broad variant classes:

- **arch variants:** compiled native binaries tied to a platform and backend;
- **no-arch variants:** mostly Python/JIT code such as Triton or other runtime-compiled kernels.

A simplified priority model is:

$$\text{priority}(v, e) = w_{\text{arch}} + w_{\text{version match}} + w_{\text{backend match}} + w_{\text{capability match}},$$

where native optimized variants are preferred when available.

## Kernel Versioning Model

The project uses integer versions corresponding to Git branches such as **v1**, **v2**, and so on.

The version is not intended to capture every patch release. Instead, it acts like an API compatibility channel.

Let:

$$A_v = \text{API contract for version } v.$$

The developer promises:

$$\forall c_1, c_2 \in \text{branch } v, \quad A_v(c_1) = A_v(c_2).$$

In words, commits within the same version branch should not break the public kernel API.

If the API changes, the version should increase:

$$A_v \neq A_{v+1}.$$

This avoids the complexity of full semantic version resolution while preventing downstream applications from accidentally breaking when pulling from `main`.

## Unique Module Naming

Python normally cannot load two different versions of the same module name at the same time. The Kernels client solves this by assigning each loaded kernel a unique identifier.

A simplified identifier is:

$$\text{id} = \text{concat}(\text{kernel name, backend, Git short hash}).$$

For example:

`flash_attention_3_cuda_a1b2c3d.`

The backend is included because one process may need both CPU and CUDA variants of the same logical kernel.

This prevents namespace collisions in both:

- the Python module table;
- the `torch.ops` namespace.

## Code Walkthrough

### Fetching an Operator Kernel

The simplest user path is to fetch a kernel directly.

```
from kernels import get_kernel

activation = get_kernel(
    "kernels-community/activation",
```

```

        version=1
    )

y = activation.silu_and_mul(x1, x2)

```

Line-by-line interpretation:

- `from kernels import get_kernel`: imports the client entry point.
- `get_kernel(...)`: asks the client to fetch a kernel repository from the hub.
- `"kernels-community/activation"`: names the hub repository.
- `version=1`: selects the v1 compatibility branch.
- `activation.silu_and_mul(...)`: calls an exposed function implemented by the loaded extension.

Hardware behavior:

- The Python call eventually dispatches into a native extension or JIT-backed implementation.
- For an AOT CUDA variant, the extension launches a CUDA kernel.
- The kernel reads input tensors from global memory and writes output tensors.
- If the operation is fused, intermediate tensors are avoided.

## Representative CUDA Kernel for SiLU-and-Multiply

The lecture does not focus on writing this exact kernel, but a representative implementation helps connect the distribution system to GPU execution.

```

#include <cuda_fp16.h>
#include <stdint.h>

__global__ void silu_mul_kernel(
    const half* __restrict__ x,
    const half* __restrict__ gate,
    half* __restrict__ out,
    int64_t n
) {
    int64_t idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < n) {
        float xv = __half2float(x[idx]);
        float gv = __half2float(gate[idx]);
        float sig = 1.0f / (1.0f + expf(-xv));
        float y = xv * sig * gv;
    }
}

```

```

        out[idx] = __float2half(y);
    }
}

```

Indexing math:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

Each thread computes one output element:

$$\text{out}[i] = \left( \mathbf{x}[i] \cdot \frac{1}{1 + \exp(-\mathbf{x}[i])} \right) \cdot \text{gate}[i].$$

Memory access pattern:

- Consecutive threads access consecutive elements of `x`.
- Consecutive threads access consecutive elements of `gate`.
- Consecutive threads write consecutive elements of `out`.

This is coalesced if tensors are contiguous and aligned. The kernel is mostly bandwidth-bound because each element performs a small number of FLOPs relative to global memory traffic.

A typical launch wrapper would choose:

$$\text{threads per block} = 256,$$

$$\text{blocks} = \left\lceil \frac{N}{256} \right\rceil.$$

```

int threads = 256;
int blocks = (n + threads - 1) / threads;

silu_mul_kernel<<<blocks, threads>>>(
    x_ptr,
    gate_ptr,
    out_ptr,
    n
);

```

## Representative PyTorch Binding

A PyTorch extension wrapper might look like this.



```

#include <torch/extension.h>

torch::Tensor silu_and_mul(torch::Tensor x, torch::Tensor gate) {
    TORCH_CHECK(x.is_cuda(), "x must be a CUDA tensor");
    TORCH_CHECK(gate.is_cuda(), "gate must be a CUDA tensor");
    TORCH_CHECK(x.is_contiguous(), "x must be contiguous");
    TORCH_CHECK(gate.is_contiguous(), "gate must be contiguous");
    TORCH_CHECK(x.numel() == gate.numel(), "shape mismatch");

    auto out = torch::empty_like(x);
    int64_t n = x.numel();

    launch_silu_mul(
        reinterpret_cast<const half*>(x.data_ptr<at::Half>()),
        reinterpret_cast<const half*>(gate.data_ptr<at::Half>()),
        reinterpret_cast<half*>(out.data_ptr<at::Half>()),
        n
    );

    return out;
}

PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
    m.def("silu_and_mul", &silu_and_mul, "Fused SiLU and multiply");
}

```

Important details:

- TORCH\_CHECK validates assumptions before launching.
- torch::empty\_like allocates an output tensor matching the input.
- data\_ptr retrieves raw device pointers.
- The binding exposes a Python-callable function.

In the Hugging Face Kernels format, this binding logic lives in the standardized torch extension area rather than an arbitrary upstream-specific location.

## Declarative Build Configuration

A representative build.toml structure may look like this.

```

[general]
name = "activation"
version = 1
license = "Apache-2.0"
backends = ["cuda", "metal"]

```

```
[general.hub]
repo_id = "kernels-community/activation"
```

```
[torch]
src = "torch-ext"
```

```
[kernel.activation]
backend = "cuda"
src = ["cuda/activation.cu"]
dependencies = ["torch"]
```

```
[kernel.activation_metal]
backend = "metal"
src = ["metal/activation.mm"]
dependencies = ["torch"]
```

Conceptual meaning:

- The `general` section gives kernel metadata.
- The `general.hub` section tells tooling where to upload.
- The `torch` section identifies the PyTorch binding source.
- Each `kernel.*` section declares a backend-specific implementation.

The builder reads this declarative configuration and constructs the build recipe. Kernel developers are not expected to manually encode every Nix or CMake build step.

## Layer Replacement Annotation

The lecture describes a decorator-style mechanism for marking a PyTorch layer as replaceable by a hub kernel.

A representative layer could look like this:

```
import torch
from torch import nn
from kernels import use_kernel_forward_from_hub

class SiluMul(nn.Module):
    @use_kernel_forward_from_hub("silu_mul")
    def forward(self, x, gate):
        return torch.nn.functional.silu(x) * gate
```

The annotation says that this layer's forward method may be replaced by a compatible kernel layer from the hub.

This matters because a large model library can keep its model code clean. It marks extension points without hard-coding a specific kernel repository into

every forward pass.

## Kernel Mapping

A mapping connects logical layer names to kernel implementations.

```
from kernels import KernelLayer, KernelMapping

mapping = KernelMapping({
    "silu_mul": {
        "cuda": KernelLayer(
            repo_id="kernels-community/activation",
            version=1,
            layer="SiluMul"
        )
    }
})
```

This mapping can be refined by:

- device type, such as CUDA, ROCm, XPU, CPU, or Metal;
- GPU capability range, such as Ampere versus Hopper;
- purpose, such as training versus inference;
- whether `torch.compile` compatibility is required.

A simplified dispatch rule is:

$$\text{selected layer} = f(\text{logical layer name, device, capability, mode}).$$

## Kernelizing a Model

A representative usage pattern is:

```
from kernels import use_kernel_mapping, kernelize

model = MyModel()

with use_kernel_mapping(mapping):
    kernelize(
        model,
        mode="training",
        device="cuda"
    )

y = model(x)
```

The transformation modifies the model in place. The layer object remains, but its forward path is replaced with the hub-provided kernel layer.

This design avoids replacing the entire module object, which would require carefully moving parameters, buffers, and state. Instead, it replaces only the forward behavior.

## Offline and Locked Usage

Some deployment systems need all artifacts baked into a container. For example, an inference server may want to download only the model weights at runtime, not kernel binaries.

The project supports lock files.

A representative `pyproject.toml` section:

```
[tool.kernels.dependencies]
activation = { repo_id = "kernels-community/activation", version = 1 }
flash_attention_3 = { repo_id = "kernels-community/flash-attention-3", version = 1 }
```

The command:

```
kernels lock
```

creates a lock file that pins repositories to specific revisions and records hashes.

The command:

```
kernels download
```

pre-downloads artifacts so they can be included in a Docker image or other deployment package.

At runtime, code can use local loading:

```
from kernels import load_kernel

activation = load_kernel("kernels-community/activation", version=1)
```

The key difference is:

`get_kernel` → may fetch from the internet,

whereas

`load_kernel` → loads only from local locked artifacts.

## Nix-Based Reproducible Building

### Why Nix Is Used

The builder uses Nix to solve reproducibility and dependency isolation.

Nix is described as a build system based on a pure functional language. In a pure language, evaluating the same expression gives the same result:

$$f(x) = y \quad \text{always, for the same } x.$$

Modern Nix expressions cannot freely read the current time, random numbers, arbitrary environment variables, or other outside state during evaluation. This makes build recipe generation reproducible.

The generated build recipe is then executed in an isolated sandbox containing only declared dependencies.

### Difference From Docker

Docker standardizes steps, but many Dockerfiles still depend on external mutable package repositories.

A typical Docker pattern is:

```
apt-get update
apt-get install -y gcc python3-dev
```

If run a week later, the installed packages may differ.

Nix pins the dependency universe more deeply:

Nix build = pinned compiler+pinned glibc+pinned CUDA+pinned Python+pinned PyTorch+declared source

This attacks the classic problem:

“it only works on my machine” → reproducible sandbox.

### Sandbox Purity

The Nix sandbox is conceptually like starting from an empty environment and adding only explicitly declared dependencies.

$$\text{sandbox} = \emptyset + \{\text{declared dependency}_1, \dots, \text{declared dependency}_k\}.$$

This helps prevent accidental dependence on the developer’s local CUDA installation, compiler, Python packages, or system libraries.

## Binary Caching

Because Nix derivations are reproducible, intermediate dependencies can be cached.

If many kernels depend on the same PyTorch and CUDA build environment, the expensive dependencies can be reused:

$$C_{\text{build with cache}} = C_{\text{kernel compile}} + C_{\text{cache download}},$$

instead of:

$$C_{\text{build from scratch}} = C_{\text{toolchain}} + C_{\text{PyTorch}} + C_{\text{CUDA deps}} + C_{\text{kernel compile}}.$$

This is crucial because building dependencies such as PyTorch or NCCL from source would be far too expensive for ordinary kernel development.

## Modern Toolchain With Old glibc Target

The lecture highlights a subtle Linux compatibility trick.

Building on an old distribution gives old glibc compatibility but also an old toolchain. The Kernels builder instead aims for:

modern compiler and packages + old glibc compatibility target.

This allows the produced binary to run on older Linux systems while still benefiting from newer compiler capabilities.

## Build Flow From `build.toml` to Binary

The simplified build flow is:

`build.toml` → Nix recipe → sandbox with selected dependencies → generated CMake and Python project files

The builder now generates CMake files and standard Python project files inside the sandbox. This has two purposes:

- developers can use ordinary IDEs and editors;
- the Nix build and IDE build use the same generated build files, avoiding divergence.

## Local Development Workflow

Developers can use Nix in two ways:

1. run the full builder to produce uploadable variants;
2. enter a Nix development shell for a specific variant.

For example, a developer could enter an environment with a specific PyTorch and CUDA combination. Then tools such as VS Code, Zed, CLion, or shell-based workflows can see the right headers, libraries, and compiler paths.

The lecture also mentions `direnv`. With `direnv`, entering a directory can automatically activate the right development environment.

## Cross-Backend Builds

Nix does not magically give a MacBook a CUDA GPU. However, it can make it easier to compile for backends even when the local machine does not have those toolkits installed globally.

For example, a CUDA machine can build CUDA, ROCm, XPU, and CPU variants by switching sandboxes and dependencies. The machine may not be able to execute all variants, but it can compile them.

This distinguishes:

compile  $\neq$  run.

A builder can check syntax and produce binaries for multiple backends, but runtime testing still requires appropriate hardware or an emulator.

## Kernel Client Deep Dive

### Environment Detection

When `get_kernel` is called, the client probes the environment. It checks:

- PyTorch version;
- backend PyTorch was compiled for;
- CUDA, ROCm, XPU, Metal, or CPU backend details;
- operating system;
- CPU architecture;
- Python ABI;
- historical C++ ABI where relevant;

- device capability where available.

This produces an environment descriptor:

$$e = (t, b, v_b, o, a, p, c),$$

where  $t$  is torch version,  $b$  is backend,  $v_b$  is backend version,  $o$  is OS,  $a$  is architecture,  $p$  is ABI, and  $c$  is capability.

## Filtering Variants

Given hub variants:

$$\mathcal{V} = \{v_1, \dots, v_n\},$$

the client removes variants for other platforms.

For example, on Linux CUDA x86\_64, it can discard:

- macOS Metal variants;
- Windows variants;
- ARM variants if the user is on x86\_64;
- ROCm variants if the current PyTorch is CUDA;
- variants requiring incompatible PyTorch or ABI.

## Backtracking CUDA Versions

If the exact CUDA version is unavailable, the resolver may choose a compatible older variant.

For example:

$$\text{user CUDA} = 12.8, \quad \text{available variant CUDA} = 12.6.$$

If compatibility rules allow, then:

$$12.6 \preceq 12.8 \Rightarrow \text{variant acceptable.}$$

This is useful because kernel authors may not immediately publish binaries for every new CUDA point release.



## Dependency Validation

Although the project tries to make kernels standalone, some no-arch or JIT kernels need Python dependencies.

For example:

- Triton may be assumed when available through PyTorch;
- other systems such as CuTe DSL or similar JIT frameworks may need explicit checking.

The client does not become a full package manager. Instead, it validates a curated set of allowed dependencies.

## Dynamic Import

The final selected variant is dynamically imported into Python under a unique name. This avoids collisions and enables multiple versions or backends of the same logical kernel to coexist.

This is stricter than ordinary Python import behavior. Kernel code must use relative imports internally. For example:

```
from .ops import silu_and_mul
```

Rather than:

```
from activation.ops import silu_and_mul
```

The second form is fragile because the module is not imported under its ordinary repository name. It is loaded under a unique internal identifier.

## Kernel Repositories on the Hub

### Kernel Repos as a New Hub Type

The lecture explains that Hugging Face introduced kernel repositories as a first-class hub repo type, analogous to model and dataset repositories.

Previously, kernels were sometimes hosted by abusing model repos. Kernel repos now allow the hub to understand kernel-specific metadata.

A kernel repository can display:

- supported backends;
- supported operating systems;
- CPU architecture;
- CUDA capabilities;
- target hardware;

- variant files;
- usage snippets;
- kernel cards.

## Kernel Cards

Kernel cards are analogous to model cards. They are intended to answer questions such as:

- What operation does this kernel implement?
- Which models or workloads use it?
- Which hardware does it support?
- What are its assumptions about dtype, layout, and shape?
- Is it for training, inference, or both?
- Does it support `torch.compile`?
- How should users contribute?

This matters because a binary kernel is opaque if users do not read source code. Good kernel cards reduce the trust and usability gap.

## Kernels Community

The lecture describes a Hugging Face-maintained **kernels-community** organization. It hosts converted and maintained kernels such as:

- FlashAttention variants;
- mixture-of-experts kernels;
- quantization kernels;
- Mamba-related kernels;
- activation kernels;
- image or diffusion-related kernels.

The project encourages original developers and third parties to host kernels under their own namespaces when possible.

## Adoption

The lecture mentions adoption in several projects:

- Transformers;
- Diffusers;

- Text Generation Inference;
- SGLang;
- modded nanoGPT;
- external research or experimentation projects using hub-based FlashAttention retrieval.

The practical value is especially clear for CI and deployment systems that want to avoid rebuilding heavy kernels repeatedly.

## Security and Trust

### Why Security Is Harder Than Safe Tensors

Hugging Face introduced SafeTensors to reduce risks when loading model weights. But kernels are different: a kernel package may contain native code.

Native code can execute arbitrary instructions in the user process. Therefore, a kernel distribution system must consider supply-chain attacks.

### Trust Model

The lecture describes a trust model similar in spirit to `trust_remote_code`.

By default, the client should not blindly load arbitrary untrusted code. Trusted organizations can be allow-listed, and users can explicitly opt in to loading code from less trusted sources.

A simplified policy is:

$$\text{load}(k) = \begin{cases} \text{allowed,} & \text{if trusted(publisher}(k)), \\ \text{allowed with explicit user trust,} & \text{if user opts in,} \\ \text{blocked,} & \text{otherwise.} \end{cases}$$

### Code Signing

The project plans support for code signing, likely using tooling such as Sigstore Cosign.

The intended security property is:

artifact  $\rightarrow$  signed by trusted identity  $\rightarrow$  verified before loading.

This reduces the risk from compromised hub tokens because a stolen upload token alone should not be enough if the signing key is separate and short-lived.

## Supply Chain Threats

Important threats include:

- malicious kernel uploads;
- compromised maintainer accounts;
- replaced binaries;
- namespace confusion;
- accidental loading from an untrusted organization;
- vulnerable dependencies in JIT kernels.

Mitigations include:

- organization namespaces;
- trusted-by-default limited kernel sets;
- explicit trust flags;
- code signing;
- hash-based lock files;
- reproducible builds;
- metadata inspection and kernel cards.

## Performance Intuition

### Why Prebuilt Kernels Reduce Deployment Latency

For production inference, startup time matters. Consider a service that starts a container and must quickly load a model.

Let:

$$T_{\text{cold}} = T_{\text{container start}} + T_{\text{model download}} + T_{\text{kernel compile}} + T_{\text{warmup}}.$$

If kernels are prebuilt and predownloaded:

$$T_{\text{cold}} = T_{\text{container start}} + T_{\text{model download}} + T_{\text{kernel load}} + T_{\text{warmup}}.$$

Because:

$$T_{\text{kernel compile}} \gg T_{\text{kernel load}},$$

AOT distribution can significantly improve cold-start latency.

## Why CI Gets Faster

CI systems often rebuild kernels repeatedly. For large kernels, the compile step dominates.

A hub-distributed prebuilt kernel changes the workflow:

CI : download binary  $\rightarrow$  run tests,

instead of:

CI : install toolchain  $\rightarrow$  compile extension  $\rightarrow$  run tests.

This is why SGLang-style adoption for FlashAttention-3 can speed up CI.

## Why Specialized Kernels Still Matter

High-level frameworks cannot always generate the best code automatically because model-specific details matter.

Examples include:

- attention sinks in particular model families;
- custom mixture-of-experts routing patterns;
- quantization formats with special packing;
- fixed-shape inference on edge devices;
- fused activation and projection patterns;
- model-specific causal masking or positional encoding tricks.

The more specific the workload, the more opportunity there is to reduce overhead.

## General Kernels Versus Workload-Specific Kernels

A general kernel handles many shapes and conditions:

$$K_{\text{general}}(x; \theta) \quad \text{for many } x.$$

A workload-specific kernel specializes to a narrower domain:

$$K_{\text{specialized}}(x; \theta, s_0) \quad \text{for shape or workload } s_0.$$

Specialization can improve performance by:

- removing branches;

- hard-coding tile sizes;
- reducing dynamic indexing;
- specializing shared memory layouts;
- specializing tensor-core instruction choices;
- precomputing constants;
- shrinking code paths.

But specialization increases variant count. Therefore, workload-specific kernels require careful indexing and metadata.

## Autotuning as a Social Process

The lecture ends with a discussion of more social or distributed autotuning. Autotuning is often embarrassingly parallel:

$$\Theta = \{\theta_1, \theta_2, \dots, \theta_m\},$$

where each  $\theta_i$  is a configuration such as block size, tile size, number of warps, pipeline stages, vectorization width, or memory layout.

Each configuration can be benchmarked independently:

$$\theta^* = \arg \min_{\theta \in \Theta} T(K_\theta, x).$$

A hub could eventually store not only kernel code, but also useful tuning results for hardware and workload combinations:

$$(\text{kernel}, \text{hardware}, \text{shape}) \rightarrow \theta^*.$$

This would allow one user's tuning work to reduce another user's search space.

## Common Misconceptions

### Misconception: A Kernel Is Just a Python Package

A kernel package may contain native binary code and GPU-specific instructions. It is closer to distributing a compiled extension than distributing pure Python.

The correct view is:

kernel package = Python wrapper+native binary or JIT code+metadata+compatibility constraints.

### **Misconception: JIT Is Always Better Than AOT**

JIT compilation adapts to hardware and shapes, but it introduces runtime warm-up cost.

AOT is preferable when:

- cold-start latency matters;
- deployments must be deterministic;
- local compilation is impossible or undesirable;
- CI cost matters;
- binaries can be validated before release.

JIT is preferable when:

- hardware changes quickly;
- shape specialization is essential;
- the workload runs long enough to amortize compile time;
- the compiler can generate better code from runtime constants.

### **Misconception: CUDA Version Compatibility Is Simple**

CUDA compatibility involves the driver, runtime libraries, toolkit, generated PTX/SASS, and backend libraries. A newer driver often supports older runtime code, but this does not mean every binary works against every future toolkit.

### **Misconception: Docker Alone Solves Reproducibility**

Docker helps isolate environments, but ordinary Dockerfiles may still depend on mutable package repositories. Nix pins the dependency universe more deeply.

### **Misconception: Versioning Needs Full Semantic Versioning**

For kernels, API surfaces are often small. The lecture argues that integer branch-based compatibility channels are easier for users and maintainers than full semantic version constraints.

### **Misconception: Loading Multiple Kernel Versions Is Easy in Python**

Python's module system normally allows one loaded module per name. Supporting multiple versions and backends requires unique internal names and careful torch operator registration.

## Practical Takeaways

1. **Kernel distribution is a systems problem.** It is not enough to write a fast CUDA kernel; one must ship it safely and reproducibly.
2. **Build matrices explode multiplicatively.** Every added backend, OS, architecture, ABI, and PyTorch version multiplies maintenance burden.
3. **AOT and JIT solve different problems.** AOT helps deployment and CI; JIT helps portability and specialization.
4. **Standardized input and output formats matter.** A consistent `build.toml`, source layout, and variant directory structure make automation possible.
5. **The client should handle variant resolution.** Users should not manually pick binaries based on CUDA, glibc, or PyTorch ABI details.
6. **Nix is used to make builds reproducible.** It pins compilers, CUDA, PyTorch, glibc, and other dependencies.
7. **glibc compatibility is critical for Linux binaries.** Building against an old compatibility target allows binaries to run on older production systems.
8. **Layer replacement is essential for large frameworks.** Transformers and Diffusers need clean model code with kernelization as a separate concern.
9. **Security must be explicit.** Loading native code requires trust policies, signing, lock files, and careful namespace management.
10. **Future value lies in specialization.** Model-specific and workload-specific kernels can unlock performance beyond generic kernels.

## Project or Experiment Ideas

### Experiment 1: Compare Local Build Versus Prebuilt Load

Measure the time for:

- installing and compiling a heavy extension locally;
- downloading and loading a prebuilt kernel variant.

Record:

$$T_{\text{compile}}, \quad T_{\text{download}}, \quad T_{\text{load}}.$$

The expected relationship is:

$$T_{\text{compile}} \gg T_{\text{download}} + T_{\text{load}}.$$



## Experiment 2: Fused Versus Unfused Activation

Implement:

$$y = \text{SiLU}(x) \odot g$$

in three ways:

1. PyTorch unfused operations;
2. `torch.compile`;
3. a custom CUDA or Triton fused kernel.

Measure bandwidth:

$$\text{effective bandwidth} = \frac{\text{bytes moved}}{T}.$$

Compare whether the fused implementation approaches memory bandwidth limits.

## Experiment 3: Variant Resolution Simulation

Write a Python script that models variant resolution.

```
variants = [
    {"torch": "2.10", "backend": "cuda", "cuda": "12.6", "os": "linux"},
    {"torch": "2.10", "backend": "cuda", "cuda": "12.8", "os": "linux"},
    {"torch": "2.10", "backend": "metal", "os": "darwin"},
]

env = {
    "torch": "2.10",
    "backend": "cuda",
    "cuda": "12.8",
    "os": "linux",
}

compatible = [
    v for v in variants
    if v["torch"] == env["torch"]
    and v["backend"] == env["backend"]
    and v["os"] == env["os"]
    and v.get("cuda", "0") <= env["cuda"]
]

best = sorted(compatible, key=lambda v: v.get("cuda", "0"))[-1]
print(best)
```

Extend it with GPU capability, ABI, and no-arch JIT fallback.

## Experiment 4: Reproducibility With Nix Versus Docker

Build the same simple C++ extension using:

- a mutable Dockerfile using `apt-get update`;
- a Nix-pinned environment.

Compare:

- compiler versions;
- linked glibc symbols;
- binary hashes;
- reproducibility over time.

## Experiment 5: Kernel Card Template

Design a kernel card template for a custom kernel. Include:

- operation definition;
- supported dtype;
- supported layouts;
- supported devices;
- expected input shapes;
- training or inference mode;
- known limitations;
- benchmark table;
- safety and trust notes.

## Experiment 6: Model-Specific Kernel Search

Pick a transformer model with a non-standard feature, such as attention sinks, custom rotary embeddings, or mixture-of-experts routing.

Identify whether a specialized kernel could reduce:

- memory traffic;
- kernel launches;
- branch divergence;
- temporary tensor allocation;

- communication overhead.

## Visual Concept

Draw a three-part pipeline. On the left, draw a kernel source repository containing `build.toml`, backend code, and torch extension bindings. In the center, draw the Nix-based builder producing multiple variant boxes labeled by PyTorch, backend, OS, architecture, and CUDA capability. On the right, draw the Hugging Face Hub and a Python client selecting exactly one compatible variant for the user's machine.

## Final Mental Model

Hugging Face Kernels is best understood as a **standardized supply chain for machine learning kernels**.

The kernel developer writes backend-specific code once inside a predictable structure. The Nix-based builder turns that code into reproducible, compatibility-labeled variants. The hub stores those variants as kernel repositories with metadata. The client detects the user's environment, selects the best compatible variant, validates dependencies and trust, loads it under a unique name, and exposes it to Python or model-layer replacement.

The core systems equation is:

fast ML inference = optimized kernels+correct binary compatibility+reproducible builds+safe distribution+e

The project exists because the kernel itself is only one part of performance engineering. A kernel that is impossible to build, unsafe to load, or painful to integrate will not help downstream users. The Hugging Face Kernels project tries to turn custom GPU kernels from scattered artifacts into a reusable, versioned, trusted, and automatable infrastructure layer for modern machine learning systems.